

Topic 1: Merge Sort

Question 1 (Easy) – Rank Students by Marks

A school has recently conducted a scholarship examination for all students of Class 12. The examination department has collected the marks of all students in an array. Due to the large number of participants, manually arranging the marks in ascending order has become difficult.

Your task is to write a program that takes the marks of N students and sorts them in ascending order using the Merge Sort algorithm.

Merge Sort works by repeatedly dividing the array into smaller subarrays until each subarray contains a single element. These smaller subarrays are then merged in a sorted manner to produce the final sorted array.

The school administration wants the marks to be displayed in increasing order so they can easily identify the lowest and highest scoring students and generate performance reports.

Input

- First line contains an integer N , representing the number of students.
- Second line contains N space-separated integers, representing the marks of the students.

Output

- Print the marks in ascending order.

Constraints

- $1 \leq N \leq 10^5$
- $0 \leq \text{Marks} \leq 100$

Example

Input:

6
78 45 92 34 67 81

Output:

34 45 67 78 81 92

```
#include <stdio.h>
#include <conio.h>
```

```
void merge (int a[], int low, int mid,
            int high)
```

```
{
```

```
    int i, j, k, temp[100];
```

```
    i = low;
```

```
    j = mid + 1;
```

```
    k = low;
```

```
    while (i <= mid && j <= high)
```

```
{
```

```
    if (a[i] <= a[j])
```

```
        temp[k++] = a[i++];
```

```
    else
```

```
        temp[k++] = a[j++];
```

```
}
```

```
    while (i <= mid)
```

```
        temp[k++] = a[i++];
```

```
    while (j <= low high)
```

```
        temp[k++] = a[j++];
```

```
    for (i = low; i <= high; i++)
        a[i] = temp[i];
```

```
}
```

```
void mergesort (int a[], int low,  
int high)
```

```
{
```

```
    int mid;
```

```
    if (low < high)
```

```
    {
```

```
        mid = (low + high) / 2;
```

```
        mergesort (a, low, mid);
```

```
        mergesort (a, mid+1, high);
```

```
        merge (a, low, mid, high);
```

```
    }
```

```
}
```

```
void main()
```

```
{ int a[100], n, i;
```

```
  clrscr();
```

```
  printf("enter no. of students: ");
```

```
  scanf("%d", &n);
```

```
  printf("enter student marks:\n");
```

```
  for (i = 0; i < n; i++)
```

```
    scanf("%d", &a[i]);
```

```
  mergesort (0, 0, n-1);
```

printf ("marks in ascending order: %n");

for (i = 0; i < n; i++)
printf ("%d", a[i]);

getch();

}


```
#include<stdio.h>
#include<conio.h>
```

```
void merge (int a[],int low,int mid ,int high)
```

```
{
```

```
    int i,j,k,temp[100];
```

```
    i=low;
```

```
    j=mid+1;
```

```
    k=low;
```

```
    while(i<=mid && j<=high)
```

```
    {
```

```
        if(a[i]<=a[j])
```

```
            temp[k++]=a[i++];
```

```
        else
```

```
            temp[k++]=a[j++];
```

```
    }
```

```
    while(i<=mid)
```

```
57:46
```

```
while(i<=mid)
    temp[k++]=a[i++];
while(j<=high)
    temp[k++]=a[j++];
for(i=low; i<=high;i++)
    a[i]=temp[i];
}

void mergesort (int a[],int low,int high)
{
    int mid;

    if (low< high)
    {
        mid =( low+high)/2;

        mergesort(a, low,mid);
        mergesort(a,mid+1,high);

        merge(a, low,mid,high);
    }
}
```



```
    }  
}  
void main()  
{  
    int a[100],n,i;  
  
    clrscr();  
    printf("enter number of students:");  
    scanf("%d",&n);  
  
    printf("enter students marks:\n");  
    for(i=0;i<n;i++)  
        scanf("%d",&a[i]);  
  
    mergesort (a,0,n-1);  
  
    printf(" marks in ascending order:\n");  
    for(i=0;i<n;i++)  
        printf("%d",a[i]);  
  
    getch();  
}
```

57:46

= File Edit Search Run Compile Debug Project Option

[■]

VASHUU.C

```
    getch();  
}
```

*

57:46

◀

Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9

enter number of students:6

enter students marks:

78 45 92 34 67 81

marks in ascending order:

34 45 67 78 81 92 _

Question 2 (Easy) – Organizing Product Prices

An e-commerce company stores product prices in the order they are added to the platform. Before publishing a seasonal sales report, the company wants all product prices to be arranged in ascending order.

Since the number of products can be very large, an efficient sorting technique is required. Your task is to implement Merge Sort to arrange the prices from the cheapest product to the most expensive.

The program should divide the list into smaller parts, sort each part, and merge them back together while maintaining the correct order.

Input

- First line contains integer N .
- Second line contains N space-separated product prices.

Output

- Print the sorted prices in ascending order.

Constraints

- $1 \leq N \leq 100000$
- $1 \leq \text{Price} \leq 1000000$

Example

Input:

5
2500 1500 4000 3000 1000

Output:

1000 1500 2500 3000 4000



```
# include <stdio.h>
```

```
# include <conio.h>
```

```
void merge (int a[], int low, int mid,  
            int high)
```

```
{
```

```
    int i, j, k temp [100];
```

```
    i = low;
```

```
    j = mid + 1;
```

```
    k = low;
```

```
    while (i <= mid && j <= high)
```

```
    {
```

```
        if (a[i] <= a[j])
```

```
            temp[k++] = a[i++];
```

```
        else
```

```
            temp[k++] = a[j++];
```

```
}
```



```
while (i <= mid)
    temp[k++] = a[j++];
```

```
while (j <= high)
    temp[k++] = a[j++];
```

```
for (i = low; i <= high; i++)
    a[i] = temp[i];
```

```
}
```

```
void mergesort (int a[], int low,
                int high)
```

```
{
```

```
    int mid;
```

```
    if (low < high)
```

```
{
```

```
        mid = (low + high) / 2;
```

```
        mergesort(a, low, mid);
```

```
        mergesort(a, mid + 1, high);
```

```
        merge(a, low, mid, high);
```

```
}
```

```
}
```

```
void main()
{
    int a[100], n, i;
    clrscr();

    printf("enter no. of products:");
    scanf("%d", &n);
printf("%d", n);
    printf("Enter product prices:
    \n");

    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    mergesort(a, 0, n-1);

    printf("sorted prices are: \n");
    for (i = 0; i < n; i++)
        printf("%d", a[i]);

    getch();
}
```

```
#include<stdio.h>
#include<conio.h>
```

```
void merge( int a[], int low, int mid, int high)
```

```
{
```

```
int i,j,k, temp[100];
```

```
    i=low;
```

```
    j=mid+1;
```

```
    k=low;
```

```
    while(i<=mid && j<= high)
```

```
    {
```

```
        if(a[i] <= a[j])
```

```
            temp[k++] = a[i++];
```

```
        else
```

```
            temp[k++] = a[j++];
```

```
    }
```

```
    while(i<= mid)
```

```
        temp[k++] = a[i++];
```

```
    while(j<= high)
```

1:4


```
[■]
while(j<= high)
    temp[k++] =a[j++];
for(i=low; i<=high; i++)
    a[i] =temp[i];
}

void mergesort( int a[], int low,int high)
{
    int mid;
    if(low< high)
    {
        mid = (low + high) /2;

        mergesort(a, low, mid);
        mergesort(a,mid+1, high);

        merge(a, low, mid ,high);
    }
}

void main()
{
    int a[100],n,i;
```

* 41:4


```
int a[100],n,i;
clrscr();
```

```
printf("enter number of products:");
scanf("%d",&n);
```

```
printf("enter product prices:\n");
for(i=0;i<n;i++)
    scanf("%d",&a[i]);
```

```
mergesort(a,0,n-1);
```

```
printf("\nsorted prices are:\n");
for(i=0; i<n; i++)
    printf("%d ",a[i]);
```

```
getch()
```

```
}
```

61:4

enter number of products:5

enter product prices:

2500 1500 4000 3000 1000

sorted prices are:

1000 1500 2500 3000 4000 _

Question 3 (Medium) – Sort Employee Records by Salary

A multinational company has employee salary records stored in an array. Before generating annual reports, the Human Resources department wants the salary records arranged in ascending order.

The company expects the number of employees to grow significantly over time, so an efficient sorting algorithm must be used. You are required to implement Merge Sort and display the salaries in increasing order.

Additionally, after sorting, the company wants to know:

1. The lowest salary.
2. The highest salary.
3. The difference between the highest and lowest salary.

Input

- First line contains integer N .
- Second line contains N space-separated salaries.

Output

- First line: Sorted salaries.
- Second line: Lowest salary.
- Third line: Highest salary.
- Fourth line: Salary difference.

Constraints

- $1 \leq N \leq 100000$
- $1000 \leq \text{Salary} \leq 1000000$

Example

Input:

```
7
45000 25000 70000 50000 30000 90000 60000
```

Output:

```
25000 30000 45000 50000 60000 70000 90000
Lowest Salary: 25000
Highest Salary: 90000
Difference: 65000
```



③

```
#include <stdio.h>
#include <conio.h>
```

```
void merge (int a[], int low, int  
mid, int high)
```

```
{
```

```
int i, j, k;
```

```
int temp[100];
```

```
i = low;
```

```
j = mid + 1;
```

```
k = low
```

while ($i \leq \text{mid}$ & $j \leq \text{high}$)

{

if ($a[i] \leq a[j]$)

temp[k++] = a[i++];

else

temp[k++] = a[j++];

}

while ($i \leq \text{mid}$)

temp[k++] = a[i++];

while ($j \leq \text{high}$)

temp[k++] = a[j++];

for ($i = \text{low}; i \leq \text{high}; i++$)

a[i] = temp[i];

}

void mergesort (int a[], int low,
int high)

{

int mid;

if ($\text{low} < \text{high}$)

mid = $(\text{low} + \text{high}) / 2$;

```
mergeSort(a, low, mid);  
mergeSort(a, mid + 1, high);  
merge(a, low, mid, high);  
}
```

```
}
```

```
void main()  
{  
    int a[100], n, i;  
    int lowest, highest, diff;  
    clrscr();
```

```
    printf("enter no. of employee:");  
    scanf("%d", &n);
```

```
    printf("enter salaries: \n");  
    for (i = 0; i < n; i++)  
        scanf("%d", &a[i]);
```

```
    lowest = a[0];  
    highest = a[n-1];  
    diff = highest - lowest;
```

```
    printf("In lowest salary: %.d",  
        lowest);
```


Date: _____
Page: _____

print ("In highest salary: %.d",
highest);

print ("In Difference: %.d", diff);

getch();

}

```
#include<stdio.h>
#include<conio.h>
```

```
void merge(long int a[], int low, int mid, int high)
```

```
{
    int i,j,k;
    long int temp[100];
```

```
    i=low;
    j=mid+1;
    k=low;
```

```
    while(i<=mid && j<= high)
```

```
    {
        if(a[i] <= a[j])
            temp[k++]=a[i++];
        else
            temp[k++] = a[j++];
    }
```

```
    while(i<= mid)
        temp[k++] = a[i++];
```



```
    temp[k++] = a[i++];
while(j<= high)
    temp[k++] = a[j++];
for(i=low; i<=high; i++)
    a[i] = temp[i];
}
void mergesort(long int a[], int low,int high)
{
    int mid;
    if(low< high)
    {
        mid = (low + high) /2;

        mergesort(a, low, mid);
        mergesort(a,mid+1, high);

        merge(a, low, mid ,high);
    }
}
void main()
{
```



```
{
    long int a[100];
    long int lowest, highest, diff;
    int n, i;
    clrscr();

    printf("enter number of employees:");
    scanf("%d", &n);

    printf("enter salaries:\n");
    for(i=0; i<n; i++)
        scanf("%ld", &a[i]);

    mergesort(a, 0, n-1);

    printf("\nsorted salaries:\n");
    for(i=0; i<n; i++)
        printf("%ld ", a[i]);

    lowest=a[0];
    highest=a[n-1];
```

66:29

```
highest=a[n-1];  
diff=highest - lowest;  
  
printf("\nlowest salary: %ld", lowest);  
printf("\nhighest salary:%ld",highest);  
printf("\ndifference:%ld",diff);  
  
getch();  
}
```

enter number of employees:7

enter salaries:

25000 30000 45000 50000 70000 60000 90000

sorted salaries:

25000 30000 45000 50000 60000 70000 90000

lowest salary: 25000

highest salary:90000

difference:65000

Topic 2: Quick Sort

Question 1 (Easy) – Race Timing Analysis

A sports academy records the completion times (in seconds) of runners participating in a race. The coach wants to arrange the timings from the fastest runner to the slowest runner.

To process large datasets efficiently, the academy has decided to use the Quick Sort algorithm. Your task is to sort the race timings in ascending order.

Input

- First line contains integer N .
- Second line contains N space-separated timings.

Output

- Print the timings in ascending order.

Constraints

- $1 \leq N \leq 100000$
- $1 \leq \text{Time} \leq 10000$

Example

Input:

6
55 49 62 58 45 51

Output:

45 49 51 55 58 62

Q1.

```
# include <stdio.h>
# include <conio.h>
```

```
void quicksort (int a[], int low, int  
high)
```

```
{
```

```
    int i, j, pivot, temp;
```

```
    if (low < high)
```

```
{
```

```
    pivot = a [low];
```

```
    i = low;
```

```
    j = high;
```

```
    while (i < j)
```

```
{
```

```
while (a[i] <= pivot && i < high)
```

```
    i++;
```

```
while (a[j] > pivot)
```

```
    j--;
```

```
if (i < j)
```

```
{
```

```
    temp = a[i];
```

```
    a[i] = a[j];
```

```
    a[j] = temp;
```

```
}
```

```
}
```

```
a[low] = a[j];
```

```
a[j] = pivot;
```

```
quicksort(a, low, i-1);
```

```
quicksort(a, i+1, high);
```

```
}
```

```
}
```

```
void main()
```

```
{
```

```
    int a[100], n, i;
```


Date _____
Page _____

```
clear();
```

```
printf("enter no. of timings: ");
```

```
scanf("%d", &n);
```

```
printf("enter timings: \n");
```

```
for (i = 0; i < n; i++)
```

```
    scanf("%d", &a[i]);
```

```
quicksort(a, 0, n-1);
```

```
printf("In sorted timings: \n");
```

```
for (i = 0; i < n; i++)
```

```
    printf("%d", a[i]);
```

```
getch();
```

```
}
```

File Edit Search Run Compile Debug Project Options Window Help
ABHISHEK.CPP 5=

```
#include<stdio.h>  
#include<conio.h>
```

```
void quicksort (int a[],int low ,int high)  
{
```

```
    int i,j,pivot,temp;
```

```
    if(low<high)
```

```
    {
```

```
        pivot =a [low];
```

```
        i=low;
```

```
        j=high;
```

```
        while (i<j)
```

```
        {
```

```
            while (a[i] <= pivot && i<high)
```

```
                i++;
```

```
            while (a[j]> pivot)
```

```
                j--;
```

```
            if (i<j)
```

```
            {
```

18:17

F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 Make F10 Men

```
[■]
    if (i<j)
    {
        temp=a[i];
        a[i]=a[j];
        a[j]=temp;
    }
}

    a[low]=a[j];
    a[j]=pivot;

    quicksort (a,low,j-1);
    quicksort (a,j+1,high);
}

void main()
{
    int a[100],n,i;

    clrscr();

    printf("enter number of timings:");
```

40:17


```
{  
    int a[100],n,i;  
  
    clrscr();  
  
    printf("enter number of timings:");  
    scanf("%d",&n);  
  
    printf("enter timings:\n");  
    for(i=0;i<n;i++)  
        scanf("%d",&a[i]);  
  
    quicksort (a,0,n-1);  
  
    printf("\nsorted timings:\n");  
    for(i=0;i<n;i++)  
  
        printf("%d ", a[i]);  
    getch();  
}
```

55:17

enter number of timings:6

enter timings:

55 49 62 58 45 51

sorted timings:

45 49 51 55 58 62 _

Question 2 (Easy) – Organizing Library Book IDs

A library stores books using unique identification numbers. Due to years of additions and removals, the IDs are no longer arranged properly.

The librarian wants all IDs sorted in ascending order before migrating the data to a new management system. Implement Quick Sort to arrange the IDs efficiently.

Input

- First line contains integer N .
- Second line contains N space-separated book IDs.

Output

- Print the sorted IDs.

Constraints

- $1 \leq N \leq 100000$
- $1 \leq ID \leq 10^9$

Example

Input:

```
5
1025 1001 1050 1010 1005
```

Output:

```
1001 1005 1010 1025 1050
```



```
# include <stdio.h>  
# include <conio.h>
```

```
void quickSort (int a[], int low, int  
high)
```

```
{
```

```
int i, j, pivot, temp;
```

```
if (low < high)
```

```
{
```

```
    pivot = a[low];
```

```
    i = low;
```

```
    j = high;
```

```
    while (i < j)
```

```
    {
```

```
        while (a[i] <= pivot && i < high)
```

```
            i++;
```

```
        while (a[j] > pivot)
```

```
            j--;
```

```
        while (a[i] < pivot)
```

```
            if (i < j)
```

```
            {
```

```
                temp = a[i];
```

```
                a[i] = a[j];
```

```
                a[j] = temp;
```

```
            }
```

```
    }
```

```
    a[low] = a[i];
```

```
    a[j] = pivot;
```

```
quicksort (a, low, j-1);  
quicksort (a, j+1, high);
```

```
}
```

```
}
```

```
void main()  
{  
    int a[100], n, i;  
    clrscr();  
  
    printf ("enter no. of books IDs:");  
    scanf ("%d", &n);  
  
    printf ("enter IDs: \n");  
    for (i=0; i<n; i++)  
        scanf ("%d", &a[i]);  
  
    quicksort (a, 0, n-1);  
  
    printf ("Insorted IDs: \n");  
    for (i=0; i<n; i++)  
        printf ("%d", a[i]);  
  
    getch();  
}
```


File Edit Search Run Compile Debug Project Options Window
[■] ABHISHEK.CPP 5

```
#include<stdio.h>  
#include<conio.h>
```

```
void quicksort (int a[],int low ,int high)  
{
```

```
    int i,j,pivot,temp;
```

```
    if(low<high)
```

```
    {
```

```
        pivot =a [low];
```

```
        i=low;
```

```
        j=high;
```

```
        while (i<j)
```

```
        {
```

```
            while (a[i] <= pivot && i<high)
```

```
                i++;
```

```
            while (a[j]> pivot)
```

```
                j--;
```

```
            if (i<j)
```

```
            {
```

18:17

F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 Make F10 M

```
[■]
if (i<j)
{
    temp=a[i];
    a[i]=a[j];
    a[j]=temp;
}

a[low]=a[j];
a[j]=pivot;

quicksort (a,low,j-1);
quicksort (a,j+1,high);
}

void main()
{
    int a[100],n,i;

    clrscr();

    printf("enter number of timings:");
```

40:17


```
int a[100],n,i;

clrscr();

printf("enter number of books ids:");
scanf("%d",&n);

printf("enter ids:\n");
for(i=0;i<n;i++)
scanf("%d",&a[i]);

quicksort (a,0,n-1);

printf("\nsorted ids:\n");
for(i=0;i<n;i++)

printf("%d ", a[i]);

getch();
}
```


enter number of books ids:5

enter ids:

1025 1001 1050 1010 1005

sorted ids:

1001 1005 1010 1025 1050 _

Question 3 (Medium) – Online Store Sales Ranking

An online marketplace tracks the daily sales made by different sellers. At the end of each week, management wants to identify the best-performing sellers.

You are given the total sales made by N sellers. Using Quick Sort, arrange the sales figures in descending order.

After sorting, display:

1. Sorted sales figures.
2. Top 3 highest sales values.
3. Average of the top 3 sales values.

Input

- First line contains integer N .
- Second line contains N space-separated sales values.

Output

- Sorted sales figures in descending order.
- Top 3 sales values.
- Average of top 3 sales.

Constraints

- $3 \leq N \leq 100000$
- $1 \leq \text{Sales} \leq 1000000$

Example

Input:

6
1200 5000 3000 2500 7000 4500

Output:

7000 5000 4500 3000 2500 1200
Top 3: 7000 5000 4500
Average: 5500

#include <stdio.h>

#include <conio.h>

void quicksort (int a[], int low,
int high)

{
int i, j, pivot, temp;

if (low < high)

{
pivot = a[low];
i = low;
j = high;

while (i < j)

{
~~while (a[j] < pivot)~~
while (a[i] >= pivot && i < high)
i++;
while (a[j] < pivot)
j--;

if (i < j)

{
temp = a[i];
a[i] = a[j];
a[j] = temp;

}

}

```
a[low] = a[j];
```

```
a[j] = pivot;
```

```
quicksort(a, low, j-1);
```

```
quicksort(a, j+1, high);
```

}

}

```
void main()
```

```
{ int a[100], n, i, avg;
```

```
clrscr();
```

```
printf("enter no. of sales: ");
```

```
scanf("%d", &n);
```

```
printf("enter sales: \n");
```

```
for (i = 0; i < n; i++)
```

```
scanf("%d", &a[i]);
```

```
quicksort(a, 0, n-1);
```

```
printf("Sorted Sales (Descending)\n");
```

```
for (i=0; i<n; i++)  
    printf ("%d", a[i]);
```

```
printf ("In Top 3 sales : %d %d %d",  
        a[0], a[1], a[2]);
```

```
avg = (a[0] + a[1] + a[2])/3;
```

```
printf ("In Average : %d", avg);
```

```
getch();
```

3

```
#include<stdio.h>
#include<conio.h>
```

```
void quicksort (int a[],int low ,int high)
```

```
{
```

```
    int i,j,pivot,temp;
```

```
    if(low<high)
```

```
    {
```

```
        pivot =a [low];
```

```
        i=low;
```

```
        j=high;
```

```
        while (i<j)
```

```
        {
```

```
            while (a[i] >= pivot && i<high)
```

```
                i++;
```

```
            while (a[j]< pivot)
```

```
                j--;
```

```
            if (i<j)
```

```
            {
```

10:58

F1 Help

Alt-F8 Next Msg

Alt-F7 Prev Msg

Alt-F9 Compile

F9 Make

F10 Menu


```
{  
    temp=a[i];  
    a[i]=a[j];  
    a[j]=temp;  
}  
}  
    a[low]=a[j];  
    a[j]=pivot;  
  
    quicksort (a,low,j-1);  
    quicksort (a,j+1,high);  
}
```

```
void main()
```

```
{  
    int a[100],n,i,avg;
```

```
    clrscr();
```

```
    printf("enter number of sales:");
```

```
    scanf("%d",&n);
```

```
10:58
```

scanf ("%d",&n);

printf ("enter sales:\n");

for (i=0; i<n; i++)

scanf ("%d",&a[i]);

quicksort (a,0,n-1);

printf ("\nsorted sales (descending):\n");

for (i=0; i<n; i++)

printf ("%d ", a[i]);

printf ("\ntop 3 sales:%d %d %d",a[0],a[1],a[2]);

avg=(a[0]+a[1]+a[2])/3;

printf ("\naverage: %d",avg);

getch();

}

10:58

F1 Help

Alt-F8 Next Msg

Alt-F7 Prev Msg

Alt-F9 Compile

F9 Make

F10 Menu

enter number of sales:6

enter sales:

1200 5000 3000 2500 7000 4500

sorted sales (descending):

7000 5000 4500 3000 2500 1200

top 3 sales:7000 5000 4500

average: 5500_